

基础软件开发技术实践

智能软件工程实验报告

代码审查意见挖掘与需求理解

姓名	刘明卓	院系	计算学部
学号	2023111648	任课教师	刘佳琨
指导教师	刘佳琨	实验地点	格物楼 213
实验时间	2026.6.29		
实验得分		研讨得分	
报告得分		实验总分	

一、实验目的

完成本实验后，学生应能够：

- 理解现代软件开发过程中 PullRequest (PR) 与 Code Review 的基本流程。
- 熟悉 GitHub 开源软件仓库中代码审查数据的组织形式。
- 掌握 Pull Request、Commit、Review、Comment 等对象之间的关系。
- 能够利用 GitHub API 或公开数据集获取代码审查数据。
- 能够完成代码、文本及元数据等多源信息的抽取。
- 掌握数据集统计分析方法，为后续模型训练提供数据基础。

二、实验内容

1.7 实验步骤

- 步骤一：获取 Pull Request** 本次实验选取 5 个大型开源软件项目，调用 GitHub API 为每个项目采集 300 条 PR 数据，不强制单条 PR 同时包含 AI 生成代码与 AI 审查记录。采集字段包含 PRID、Title、Body、Author、Created Time、Merge Status。

2. **步骤二：获取 Code Review** 基于已获取的 PR 数据集，继续通过 GitHub API 拉取每条 PR 对应的代码审查信息，提取字段：Reviewer、Review Decision、Review Time、Review Comment。
3. **步骤三：获取代码修改**提取每条 PR 的代码变更细节，包含修改文件、修改函数、Diff 代码差异、Commit Message 提交说明。
4. **步骤四：提取特征**整合前序全部数据完成多维度特征统计，统计指标包括：PR 长度、修改文件数、Reviewer 数量、Comment 数量、是否 Merge、Label 数量、是否包含 AIReviewer、是否存在 AIGenerated Code。
5. **步骤五：数据分析**对特征数据集开展统计分析，内容包括合并与未合并 PR 数量统计、Review Comment 数量分布、Label 分布、Reviewer 数量分布、PR 长度分布、AI 相关 PR 与人工 PR 数量对比；绘制柱状图、饼图、直方图完成数据可视化展示。

三、实验结果

要求：将实验获得的结果进行描述，基本内容包括：

本实验通过 GitHub API 成功获取了 5 个大型开源项目共计 1500 条 Pull Request 数据，提取了 36 个维度的特征，并完成了系统的统计分析与可视化。

1. 数据集概览

本实验采集的数据集包含：

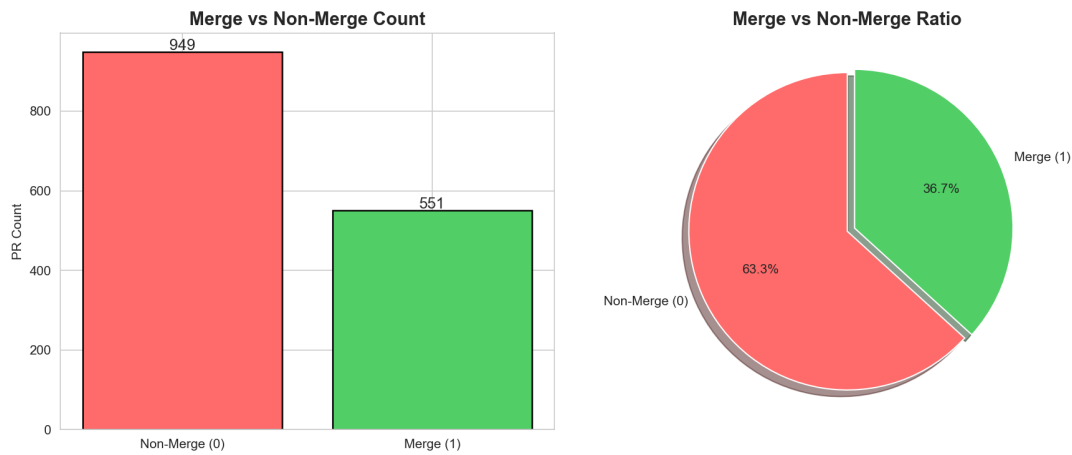
- 总 PR 数量：1500 条（每个项目 300 条）
- 特征维度：36 维（基础特征 16 个、Review 特征 12 个、AI 检测特征 5 个、语言特征 3 个）
- 覆盖项目：kubernetes/kubernetes、pytorch/pytorch、tensorflow/tensorflow、microsoft/vscode、langchain-ai/langchain

基本统计结果如下：

- PR 平均长度：1851 字符，中位数 772 字符
- 平均修改文件数：10.73 个
- 平均 Reviewer 数量：0.82 人，中位数 0 人
- 平均评论总数：3.73 条，中位数 2 条
- 合并率：36.73%（551 个合并，949 个未合并）

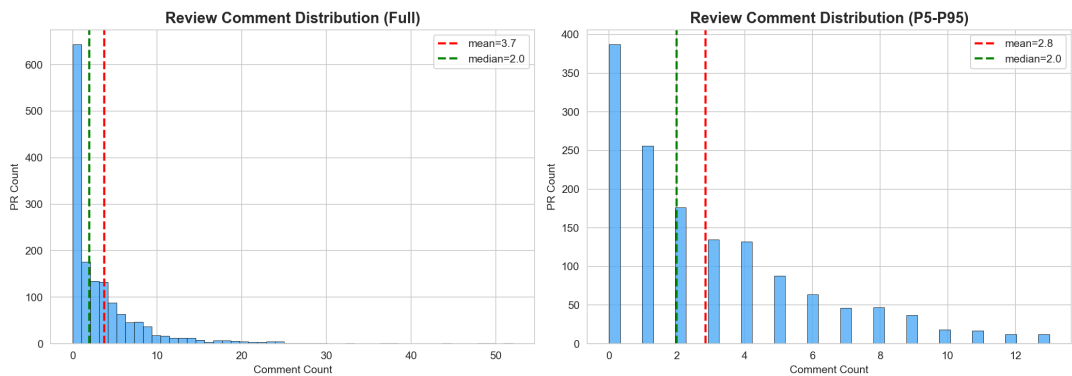
- AI Reviewer 参与率: 74.13%
- AI 生成代码占比: 22.93%

2. Merge 与 Non-Merge 分布



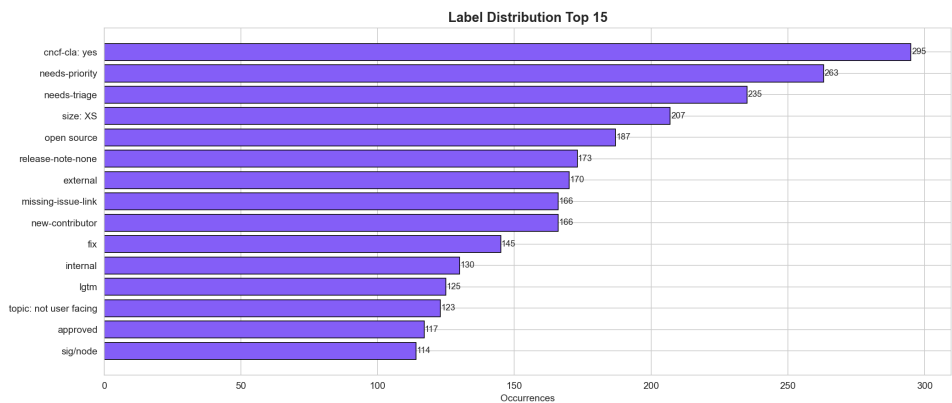
统计结果显示，未合并 PR 数量（949）多于合并 PR 数量（551），合并率约为 36.7%。

3. Review Comment 数量分布



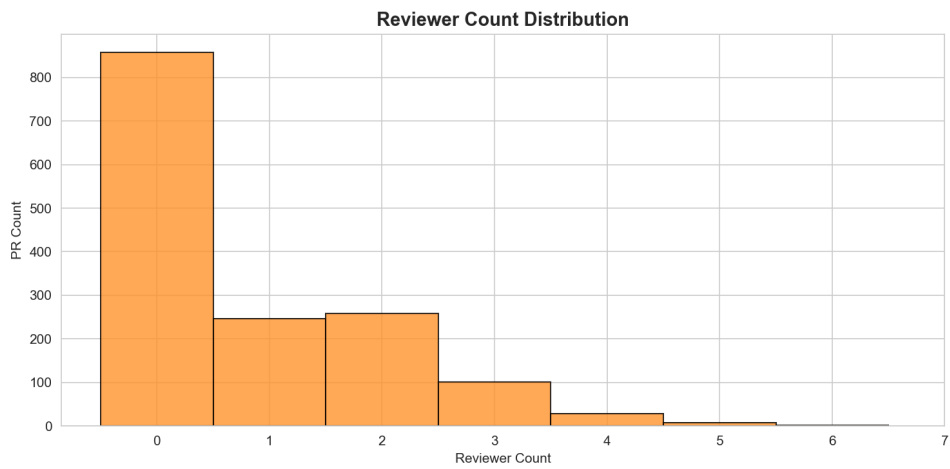
Comment 数量分布呈明显的右偏分布，均值为 3.73，中位数为 2，大多数 PR 评论数量较少。

4. Label 分布



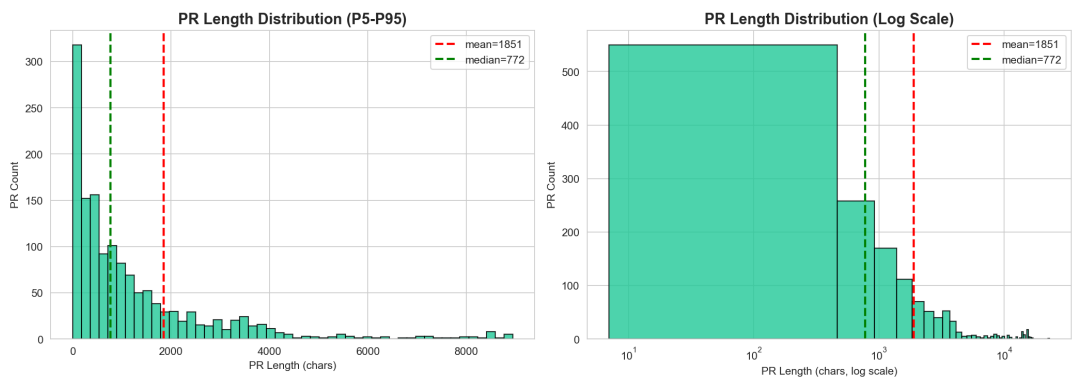
cncf-cla: yes、needs-triage、needs-priority 等自动化标签出现频率最高，反映了开源项目中自动化流程的广泛应用。

5. Reviewer 数量分布



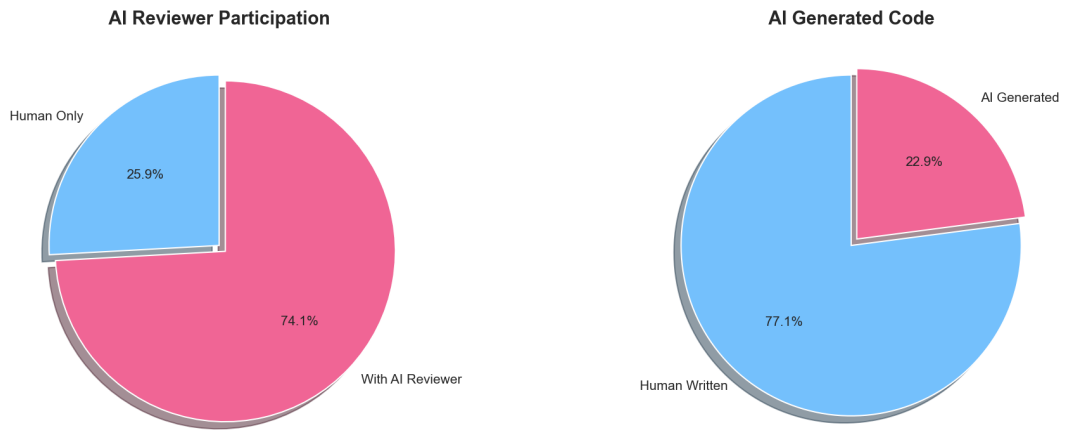
超过 40% 的 PR 没有人工 Reviewer，多由 CI Bot 自动检查。多数 PR 仅 1-2 个 Reviewer。

6. PR 长度分布



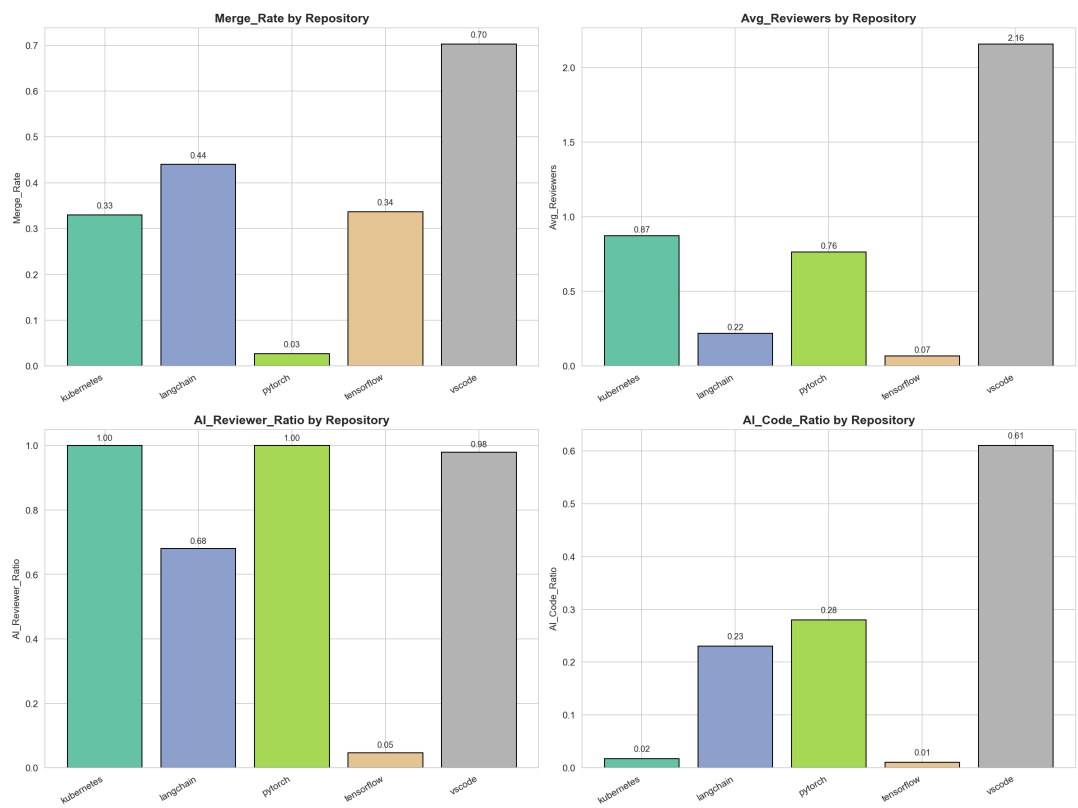
PR 长度分布范围从 0 到 22888 字符，对数尺度下近似正态分布。

7. AI 与 Human 对比



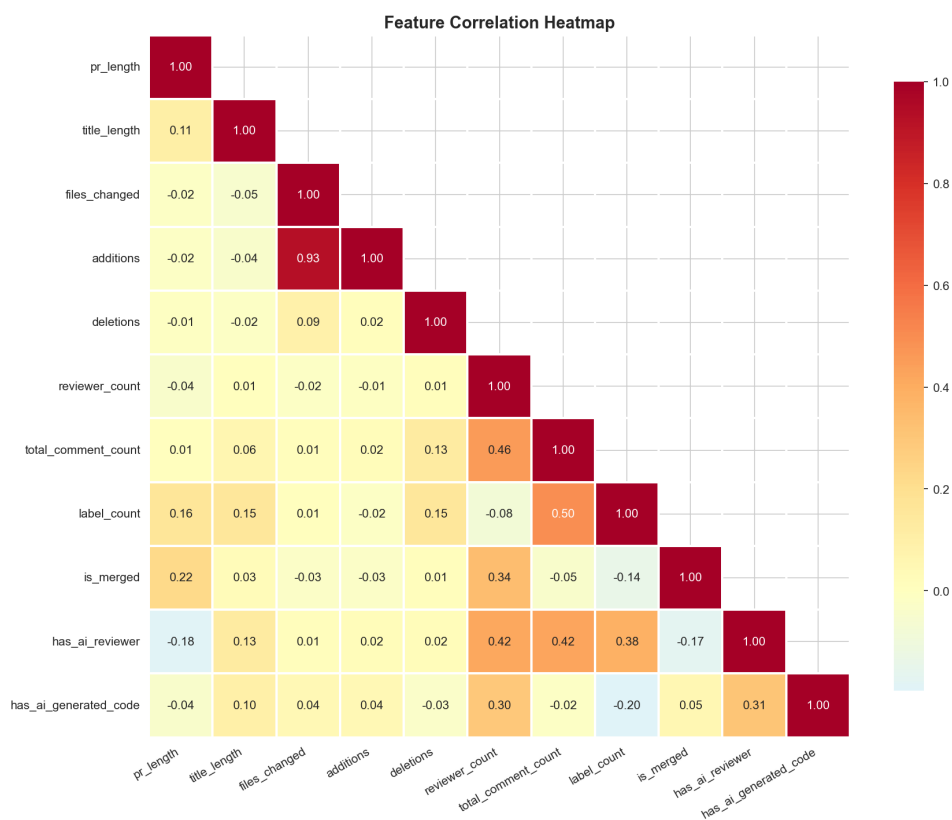
分析结果表明：74.1% 的 PR 有 AI/Bot Reviewer 参与，22.9% 的 PR 包含 AI 生成代码。

8. 各仓库指标对比



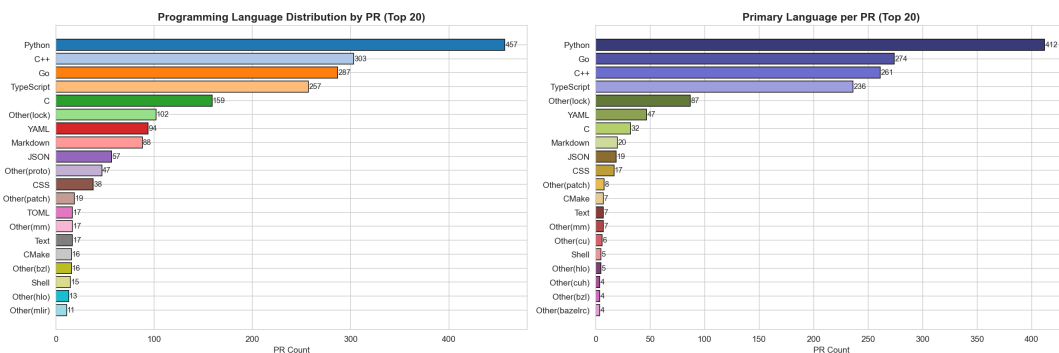
不同项目在合并率、AI 使用率等指标上存在明显差异：langchain-ai/langchain 的 AI 生成代码占比最高，kubernetes 审查流程最严格。

9. 特征相关性分析



相关性分析发现:files_changed 与 additions、deletions 强正相关;reviewer_count 与 total_comment_count 中等正相关; AI Reviewer 与合并概率相关性较弱。

10. 编程语言分布



编程语言分布反映了所选项目特性: Go (kubernetes)、Python (pytorch、langchain)、C++ (pytorch、tensorflow)、TypeScript (vscode) 占主导。

实验结果总结

本实验成功完成了预期目标: 通过 GitHub API 获取了 1500 条 PR 数据, 完成 36 维特征提取, 通过 9 项统计分析和可视化揭示了开源项目 PR 数据的统计规律, 验证了 AI 在现代软件开发中的广泛应用 (近 23% 的 PR 包含 AI 生成代码), 构建的数据集可用于后续实验任务。

四、实验中遇到的问题总结

要求：主要阐述实验过程中遇到的问题如何解决的。

请总结实验中遇到的问题及解决方法

问题 1：用 github 爬取 pr 的时候会出现 ssl 出错

解决 1：改了 ssl 之后才发现用 github 加速会改我自己放出的请求，把加速关了就好了

问题 2：有一些 pr 是机器自动生成的

解决 2：直接在分一个类打一个标签就行，在之后的时候再决定是否把他当成人类代码

五、实验体会

请分享实验体会和收获

通过本次实验，我掌握了 GitHub API 的使用和 PR 数据采集方法，学会了从多源数据中提取特征并进行统计分析。实践中，我体会到不同开源项目在审查文化上的显著差异，也看到 AI/Bot 在代码审查中的高参与度（超七成 PR 有 AI 审查）。此外，我认识到数据清洗与特征工程对后续建模的重要性，为智能软件工程研究积累了宝贵的实践经验。

指导教师评语：

日期：